MICRO SUN AND SOLUTIONS PLC

Deresegn POS

Security Documentation

Threat Model & Security Controls

Document Version:	1.0
Application Version:	1.0.3+4
Date:	August 5, 2026
Classification:	Confidential
Prepared by:	Micro Sun & Solution PLC
Email:	amanuielt@mssethiopia.com
Phone:	+251 911 058 179

Deresegn POS - Security Documentation

1. Mobile Threat Modeling

1.1 Threat Identification

Threat ID	Category	Threat	Target	Likelihood	Impact
T-01	Tampering	APK modification / repackaging	Mobile binary	Medium	High
T-02	MITM	Interception of API traffic	Network layer	Low (HTTPS)	Critical
T-03	Credential Theft	Extraction of tokens from device storage	flutter_secure_storage	Low	Critical
T-04	Reverse Engineering	Decompilation of Dart AOT binary	APK/IPA	Medium	Medium
T-05	Insecure Storage	Offline queue data in shared_preferences (unencrypted)	Local storage	Medium	Medium
T-06	Session Hijacking	Stolen JWT tokens reused for unauthorized access	Token storage	Low	High
T-07	Unauthorized API Access	Direct API calls bypassing the mobile app	Backend API	Medium	High
T-08	Key Compromise	Extraction of MoR private keys from server filesystem	Server storage	Low	Critical
T-09	Logging Exposure	Sensitive data logged via LoggingInterceptor	Log output	Medium	Medium
T-10	Replay Attack	Replaying captured API requests	Network	Low	Medium

1.2 STRIDE Classification

Threat	S	T	R	I	D	E
APK Tampering	[x]					
MITM Attack	[x]	[x]				
Credential Theft	[x]	[x]				
Reverse Engineering	[x]					
Insecure Local Storage	[x]	[x]				
Session Hijacking	[x]	[x]	[x]			
Direct API Access	[x]	[x]	[x]			
Key Compromise	[x]	[x]				
Log Exposure	[x]					

(S=Spoofing, T=Tampering, R=Repudiation, I=Information Disclosure, D=Denial of Service, E=Elevation of Privilege)

2. Attack Surface Analysis

2.1 Mobile Application Attack Surface

Surface	Component	Exposure	Current Mitigation
Network	HTTPS to api.deresegn.com	All API traffic	TLS 1.2+, HTTPS-only
Local Storage (Encrypted)	flutter_secure_storage	JWT tokens, MoR credentials	AES-256 (Keystore/Keychain)
Local Storage (Unencrypted)	shared_preferences	Offline invoice queue	No encryption
Local Storage (Unencrypted)	cookie_jar (file system)	HTTP session cookies	No encryption
Application Binary	APK / IPA	Dart AOT compiled code	Default Flutter compilation
Inter-process Communication	None	No exported activities/services	N/A
Input Fields	Invoice forms, login forms	User-provided data	Basic validation
Logging	LoggingInterceptor output	Full request/response logging	Disabled in test mode only

2.2 Backend API Attack Surface

Surface	Component	Exposure	Current Mitigation
API Endpoints (Public)	`/api/company/register`, `/api/company/login`, `/api/companies`, `/api/branch/login`	No authentication required	Input validation, rate limiting (server-level)
API Endpoints (Authenticated)	All `/api/*` MoR-proxied routes	Requires Branch JWT + Branch-Id header	ResolveMorBranch middleware
File Upload	Branch creation (private_key, certificate)	File upload via multipart	File type validation, max size 10MB, permissions 0600
Database	MySQL (yegna_erp)	All persistent data	Eloquent ORM (parameterized queries), encrypted columns
Filesystem	storage/app/mor-credentials/	Private keys, certificates	File permissions 0600
MoR API Proxy	Outbound HTTPS to MoR gateway	Government API	Signed payloads (SHA-512), TLS

3. Risk Assessment

Risk ID	Risk	Likelihood	Impact	Severity
R-01	Hardcoded fallback MoR credentials in source code	High (present in code)	Medium	High
R-02	Offline queue stores invoice data unencrypted	Medium	Medium	Medium
R-03	LoggingInterceptor logs Authorization headers and full payloads	Medium	Medium	Medium

Risk ID	Risk	Likelihood	Impact	Severity
R-04	No certificate pinning for API connections	Medium	High	High
R-05	No root/jailbreak detection	Medium	Medium	Medium
R-06	No code obfuscation beyond default Flutter compilation	Medium	Low	Low
R-07	HTTP cookies stored unencrypted on filesystem	Low	Low	Low
R-08	Branch TIN not unique - password brute-force across branches	Low	Medium	Medium
R-09	No input sanitization on MoR payloads before signing	Low	Medium	Low
R-10	Company listing endpoint (`/api/companies`) exposes names publicly	Low	Low	Low

4. Security Controls

4.1 Authentication Controls

Control	Implementation	Status
JWT-based authentication	`tymon/jwt-auth` for company and branch	? Implemented
Password hashing	bcrypt via `Hash::make()`	? Implemented
Branch JWT + Branch-Id cross-validation	`ResolveMorBranch` middleware	? Implemented
Token expiry detection	`JwtDecoder.isExpired()` (mobile), JWT expiry (backend)	? Implemented
Automatic token refresh	`AuthInterceptor.onError()` with refresh lock	? Implemented
Session cleanup on logout	`ConfigPreference.clearTokens()`	? Implemented

4.2 Data Protection Controls

Control	Implementation	Status
Credential encryption (mobile)	`flutter_secure_storage` (AES-256, Keystore/Keychain)	? Implemented
Credential encryption (backend DB)	Laravel `encrypted` cast (AES-256-CBC)	? Implemented
Private key file permissions	`chmod 0600` on upload	? Implemented
Password hashing	bcrypt	? Implemented
TLS transport encryption	HTTPS for all API communication	? Implemented
Audit data masking	Tokens masked, secrets removed from audit records	? Implemented

5. Secure Coding Practices

5.1 Implemented Practices

Practice	Details

Practice	Details
Parameterized queries	Eloquent ORM prevents SQL injection
Password hashing	bcrypt with automatic salt
HTTPS-only communication	All endpoints served over TLS
Secure token storage	flutter_secure_storage with platform encryption
Secrets masking in audits	clientSecret removed, tokens replaced with 'MASKED'
Hidden model attributes	`client_secret`, `api_key`, `password`, `private_key_path`, `certificate_path` marked as `$hidden`
Input validation	Laravel Validator on all endpoints with required/type constraints
Error handling	Structured error responses, no stack traces exposed to clients
Minimal permission file storage	Private keys stored with 0600 permissions

6. Authentication Security

6.1 Password Policy

- Minimum length: 6 characters (enforced by backend validator)
- Hashing algorithm: bcrypt (Laravel default)
- No complexity requirements (no uppercase, special character requirements)

6.2 Token Security

- JWT algorithm: HS256 (tymon/jwt-auth default)
- Token storage: flutter_secure_storage (encrypted)
- Token lifetime: Configurable in backend (`config/jwt.php`)
- Refresh mechanism: Dedicated `/api/refresh/token` endpoint
- Concurrent refresh prevention: Static `_refreshFuture` variable prevents duplicate refresh calls

6.3 Multi-Guard Architecture

- `auth:api` guard - Company owner authentication (CompanyUser model)
- `branch` guard - Branch operator authentication (CompanyBranch model)
- Both guards use JWT tokens but with separate user providers

7. Data Protection

7.1 Encryption at Rest

Data	Encryption Method	Key Management
Mobile credentials	AES-256 (Android Keystore / iOS Keychain)	Platform-managed
DB client_secret & api_key	AES-256-CBC (Laravel encrypted cast)	APP_KEY in .env
Passwords	bcrypt hash	Self-salted
Private key files	No additional encryption (file permissions)	File system ACL

7.2 Encryption in Transit

Connection	Protocol	Details
Mobile <-> Backend	HTTPS (TLS 1.2+)	Standard TLS, no pinning
Backend <-> MoR Gateway	HTTPS (TLS 1.2+)	Via Guzzle HTTP client

7.3 Digital Signatures

- Algorithm: RSA with SHA-512 (`OPENSSL_ALGO_SHA512`)
- Key format: PEM (loaded via `openssl_pkey_get_private()`)
- Signed data: JSON-serialized request payload
- Certificate: Base64-encoded X.509 certificate attached to each outbound request

8. Logging and Monitoring

8.1 Mobile Logging

- Library: `logger` package (Dart)
- Interceptor: `LoggingInterceptor` logs all HTTP request/response cycles
- Coverage: Full request URLs, headers (including Authorization), request bodies, response bodies
- Format: Pretty-printed JSON via `JsonEncoder.withIndent`
- Production concern: Authorization tokens and full payloads are logged in debug output

8.2 Backend Logging

- Library: Laravel `Log` facade
- Error logging: MoR service exceptions with branch context
- Audit tables:
  - `mor_login_audits` - All MoR login attempts (secrets masked)
  - `mor_invoice_audits` - All invoice registration attempts (full payload)
  - `mor_cancellation_audits` - All cancellation attempts
- Masking: `clientSecret` removed from login audit requests; `accessToken` and `refreshToken` replaced with `'MASKED'`

8.3 Monitoring Gaps

- No centralized logging aggregation
- No real-time alerting on failed authentication attempts
- No anomaly detection on API usage patterns
- No mobile crash reporting service integrated

9. Incident Response

9.1 Current Capabilities

- Audit trail: All MoR interactions are logged with full request/response data
- Token revocation: Tokens can be invalidated by updating JWT secret or deleting the user/branch record
- Credential rotation: Branch credentials (client ID, secret, API key, certificate) can be re-provisioned via branch update